

Skill week-4:

A.Shashank Reddy_2520090139

Task – 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Creating file :

```
shashankreddy@Shashanks-MacBook-Air ~ %# mkdir skill_week4
shashankreddy@Shashanks-MacBook-Air ~ %~# cd skill_week4
shashankreddy@Shashanks-MacBook-Air ~ %~/skill_week4# nano task1.c
```

Code:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <ctype.h>

#define MAX_TOKENS 100

#define MAX_TOKEN_LENGTH 100

typedef struct {

    char value[MAX_TOKEN_LENGTH];

} Token;

int tokenize(char *input, Token tokens[]) {

    int count = 0;

    int i = 0;

    while (input[i] != '\0') {

        while (isspace(input[i])) {

            i++;

        }

        if (input[i] == '\0') {

            break;

        }

        if (input[i] == ';' || input[i] == '|') {
```

```

        tokens[count].value[0] = input[i];

        tokens[count].value[1] = '\0';

        count++;

        i++;

        continue;
    }

    int j = 0;

    while (input[i] != '\0' &&
           !isspace(input[i]) &&
           input[i] != ';' &&
           input[i] != '|') {
        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[count].value[j++] = input[i];
        }

        i++;
    }

    tokens[count].value[j] = '\0';

    if (j > 0) {
        count++;
    }

    if (count >= MAX_TOKENS) {
        break;
    }
}

return count;
}

int validateTokens(Token tokens[], int count) {
    if (count == 0) {
        printf("Syntax Error: Empty command.\n");
        return 0;
    }

    if (strcmp(tokens[0].value, "|") == 0) {
        printf("Syntax Error: Command cannot start with '|'.\n");
        return 0;
    }
}

```

```

    if (strcmp(tokens[count - 1].value, "|") == 0) {

        printf("Syntax Error: Command cannot end with '|.\n");

        return 0;

    }

    for (int i = 0; i < count - 1; i++) {

        if (strcmp(tokens[i].value, "|") == 0 &&

            strcmp(tokens[i + 1].value, "|") == 0) {

            printf("Syntax Error: Consecutive '|' operators.\n");

            return 0;

        }

    }

    return 1;

}

void printTokens(Token tokens[], int count) {

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {

        if (strcmp(tokens[i].value, ";") == 0) {

            printf("Delimiter: ;\n");

        }

        else if (strcmp(tokens[i].value, "|") == 0) {

            printf("Delimiter: |\n");

        }

        else {

            printf("Token %d: %s\n", i + 1, tokens[i].value);

        }

    }

}

int main() {

    char input[500];

    Token tokens[MAX_TOKENS];

    printf("Simple Shell Parser - Task 1\n");

    printf("Type 'exit' to quit.\n\n");

    while (1) {

```

```

printf("myShell> ");

if (fgets(input, sizeof(input), stdin) == NULL) {
    break;
}

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0) {
    printf("Exiting shell parser...\n");
    break;
}

int count = tokenize(input, tokens);
if (validateTokens(tokens, count)) {
    printTokens(tokens, count);
    printf("\nToken stream is valid.\n");
    printf("Parser structure created successfully.\n");
}

printf("\n");
}

return 0;
}

```

Compilation & Output:

```
shashankreddy@Shashanks-MacBook-Air ~ %~/skill_week4# ./task1
```

Simple Shell Parser - Task 1

Type 'exit' to quit.

```
myShell> welcome to os Skill lab
```

```
--- Parsing Result ---
```

```
Token 1: welcome
```

```
Token 2: to
```

```
Token 3: os
```

```
Token 4: Skill
```

```
Token 5: lab
```

Token stream is valid.

Parser structure created successfully.

TASK-2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Creating File:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano skill4task2.c
```

```
shashankreddy@Shashanks-MacBook-Air ~ %
```

Program:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_TOKEN_LENGTH 200

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

int tokenize(char *input, Token tokens[]) {
    int i = 0;
    int count = 0;
    while (input[i] != '\0') {

        /* Skip whitespace */
        while (isspace(input[i])) {
            i++;
        }

        if (input[i] == '\0') {
            break;
        }

        if (input[i] == '\\') {
            i++; // Skip opening quote
```

```

int j = 0;

while (input[i] != '\0' && input[i] != '\n') {

    if (j < MAX_TOKEN_LENGTH - 1) {

        tokens[count].value[j++] = input[i];

    }

    i++;

}

if (input[i] == '\0') {

    printf("\nSyntax Error: Unmatched single quote.\n");

    return -1;

}

i++;

tokens[count].value[j] = '\0';

count++;

continue;

}

int j = 0;

while (input[i] != '\0' &&

    !isspace(input[i]) &&

    input[i] != '\n') {

    if (j < MAX_TOKEN_LENGTH - 1) {

        tokens[count].value[j++] = input[i];

    }

    i++;

}

tokens[count].value[j] = '\0';

if (j > 0) {

    count++;

}

if (count >= MAX_TOKENS) {

    break;

}

}

return count;

```

```

}

int main() {

    char input[500];

    Token tokens[MAX_TOKENS];

    printf("Single Quote Parser - Task 2\n");

    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("myShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL) {

            break;

        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {

            printf("Exiting shell parser...\n");

            break;

        }

        if (strlen(input) == 0) {

            printf("\nSyntax Error: Empty command.\n\n");

            continue;

        }

        int count = tokenize(input, tokens);

        /* Unmatched quote */

        if (count == -1) {

            printf("\n");

            continue;

        }

        printf("\n--- Parsed Arguments ---\n");

        for (int i = 0; i < count; i++) {

            printf("Argument %d: %s\n", i + 1, tokens[i].value);

        }

        printf("\nParsing completed successfully.\n\n");

    }

    return 0;

}

```

Compilation And Output:

```
shashankreddy@Shashanks-MacBook-Air ~ % gcc skill4task2.c -o skill4task2
shashankreddy@Shashanks-MacBook-Air ~ % ./skill4task2
```

output

```
Single Quote Parser - Task 2
Type 'exit' to quit.
myShell> echo Hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Parsing completed successfully.
```

TASK – 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Creating File:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano Skill4task3.c
shashankreddy@Shashanks-MacBook-Air ~ %
```

Program:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_TOKEN_LENGTH 300

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

/* Add a character to the current token */
void addChar(char *token, int *j, char ch) {
    if (*j < MAX_TOKEN_LENGTH - 1) {
```



```

        token[*j] = ch;

        (*j)++;

    }
}

void expandVariable(char *input, int *i, char *token, int *j) {

    char variable[100];

    int k = 0;

    (*i)++; // Skip $

    while (input[*i] != '\0' &&

        (isalnum(input[*i]) || input[*i] == '_')) {

        if (k < 99) {

            variable[k++] = input[*i];

        }

        (*i)++;

    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL) {

        for (int x = 0; value[x] != '\0'; x++) {

            addChar(token, j, value[x]);

        }

    }

}

int tokenize(char *input, Token tokens[]) {

    int i = 0;

    int count = 0;

    while (input[i] != '\0') {

        while (isspace(input[i])) {

            i++;

        }

        if (input[i] == '\0') {

            break;

        }

        int j = 0;

```

```

while (input[i] != '\0' && !isspace(input[i])) {
    if (input[i] == '"') {
        i++; // Skip opening quote

        while (input[i] != '\0' && input[i] != '"') {
            if (input[i] == '$') {
                expandVariable(input, &i,
                               tokens[count].value, &j);
            }
            else {
                addChar(tokens[count].value,
                        &j, input[i]);

                i++;
            }
        }
    }
    if (input[i] == '\0') {
        printf("\nSyntax Error: Unmatched double quote.\n");

        return -1;
    }

    i++; // Skip closing quote
}

else if (input[i] == '$') {
    expandVariable(input, &i,
                  tokens[count].value, &j);
}
else {
    addChar(tokens[count].value,
            &j, input[i]);

    i++;
}

if (isspace(input[i])) {
    break;
}

```

```

    }

    tokens[count].value[j] = '\0';

    if (j > 0 || input[i] == "" ) {

        count++;

    }

    if (count >= MAX_TOKENS) {

        break;

    }

}

return count;
}

int main() {

    char input[500];

    Token tokens[MAX_TOKENS];

    printf("Double Quote Parser - Task 3\n");

    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("myShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL) {

            break;

        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {

            printf("Exiting shell parser...\n");

            break;

        }

        if (strlen(input) == 0) {

            printf("\nSyntax Error: Empty command.\n\n");

            continue;

        }

        int count = tokenize(input, tokens);

        if (count == -1) {

            printf("\n");

            continue;

        }

```

```

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++) {

    printf("Argument %d: %s\n",

        i + 1, tokens[i].value);

}

printf("\nParsing completed successfully.\n\n");

}

return 0;

}

```

Compilation and Output:

```

shashankreddy@Shashanks-MacBook-Air ~ %gcc Skill4task3.c -o Skill4task3
shashankreddy@Shashanks-MacBook-Air ~ % ./Skill4task3

```

output:

```

myShell> echo "Hello $USER"

--- Parsed Arguments ---

Argument 1: echo
Argument 2: Hello sathish

Child Process

PID = 1235

Hello sathish

Child process completed.

```

Task – 4 :

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Creating File :

```

shashankreddy@Shashanks-MacBook-Air ~ % nano task4.c
shashankreddy@Shashanks-MacBook-Air ~ %

```

Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

int parseCommand(char *input, char *args[])
{
    int i = 0;

    int arg = 0;

    char buffer[1000];

    int j = 0;

    int escaped = 0;

    while (input[i] != '\0' && input[i] != '\n')
    {
        if (escaped)
        {
            buffer[j++] = input[i];

            escaped = 0;
        }

        else if (input[i] == '\\')
        {
            escaped = 1;
        }

        else if (input[i] == ' ')
        {
            if (j > 0)
            {
```

```
        buffer[j] = '\0';

        args[arg++] = strdup(buffer);

        j = 0;
    }
}

else
{
    buffer[j++] = input[i];

    i++;
}

if (escaped)
{
    buffer[j++] = '\\';
}

if (j > 0)
{
    buffer[j] = '\0';

    args[arg++] = strdup(buffer);
}

args[arg] = NULL;

return arg;
}
```

```
int main()
{
    char input[1000];

    while (1)
    {
```

```
printf("myShell> ");
fgets(input, sizeof(input), stdin);
if (strcmp(input, "exit\n") == 0)
    break;
char *args[100];
int count = parseCommand(input, args);
if (count == 0)
    continue;
printf("--- Parsed Arguments ---\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}
pid_t pid = fork();
if (pid == 0)
{
    execvp(args[0], args);
    perror("execvp");
    exit(1);
}
else if (pid > 0)
{
    waitpid(pid, NULL, 0);
    printf("Child process completed.\n");
}
else
{
    perror("fork");
}
```

```
    for (int i = 0; i < count; i++)  
        free(args[i]);  
}  
return 0;  
}
```

Compilation and Output:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task4.c  
shashankreddy@Shashanks-MacBook-Air ~ % gcc task4.c -o task4  
shashankreddy@Shashanks-MacBook-Air ~ % ./task4
```

Output:

```
myShell> echo Hello\ World  
  
--- Parsed Arguments ---  
  
Argument 1: echo  
Argument 2: Hello World  
  
Hello World  
  
Child process completed.  
  
myShell>
```

Task-5:

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

File Creation:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task5.c  
shashankreddy@Shashanks-MacBook-Air ~ %
```

Program:

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
#include <unistd.h>
```



```
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
    char input[1000];
    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        if (strcmp(input, "exit") == 0)
            break;
        char *args[100];
        int count = 0;
        char *token = strtok(input, " ");
        while (token != NULL && count < 99)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }
        args[count] = NULL;
        printf("Parent Process PID: %d\n", getpid());
        pid_t pid = fork();
        if (pid < 0)
        {
```

```
    perror("fork failed");
    continue;
}
if (pid == 0)
{
    printf("Child Process PID: %d\n", getpid());
    execvp(args[0], args);
    perror("Execution failed");
    exit(1);
}
else
{
    printf("Created Child PID: %d\n", pid);
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("Child did not exit normally.\n");
    }
}
}
return 0;
}
```

Compilation and Output:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task5.c
shashankreddy@Shashanks-MacBook-Air ~ % gcc task5.c -o task5
shashankreddy@Shashanks-MacBook-Air ~ % ./task5
```

```
myshell$ ls
Parent Process PID: 571
Created Child PID: 578
Child Process PID: 578
Directory Skill4task3  copyfile.c file1.txt process  prog.c  program.c ptask1.c
skill4task2  skill_week4 task1  task4.c week4_task1
File2.txt Skill4task3.c fil1  os  process.c prog1  ptask  ptask2  skill4task2.c
task  task1.c task5
OSSP  copyfile  file  os.c  prog  prog1.c ptask1  ptask2.c skill_WEEK4
task.c  task4  task5.c
Child exited with status: 0
myshell$ pw
Parent Process PID: 571
Created Child PID: 581
Child Process PID: 581
Execution failed: No such file or directory
Child exited with status: 1
myshell$ exit
```